

VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ
BRNO UNIVERSITY OF TECHNOLOGY

FAKULTA INFORMAČNÍCH TECHNOLOGIÍ
FACULTY OF INFORMATION TECHNOLOGY

SÍŤOVÉ APLIKACE A SPRÁVA SÍTÍ
NETWORK APPLICATIONS AND NETWORK ADMINISTRATION

MONITOROVÁNÍ DNS KOMUNIKACE

MONITORING OF DNS COMMUNICATION

SEMESTRÁLNÍ PROJEKT
TERM PROJECT

AUTOR PRÁCE **VÁCLAV PŘIBYL**
AUTHOR

VEDOUCÍ PRÁCE **Ing. RADEK HRANICKÝ, Ph.D.**
SUPERVISOR

BRNO 2025

Obsah

1 Úvod **2**

2 Spuštění programu **3**

3 Použité knihovny **4**

4 DNS komunikace **5**

5 Výpis programu **8**

6 Testování **10**

7 Závěr **11**

Literatura **12**

Kapitola 1

Úvod

Zadáním projektu bylo vytvořit nástroje pro sledování DNS komunikace, který jsem implementoval v programovacím jazyce `C`.

Hlavní část projektu je ztvárněna v souboru `dns_monitor.c`, který zajišťuje připojení na síťový adaptér, případně zpracování *pcap/pcapng* souboru. Z takto získaných paketů pak vypisuje do terminálu podstatné informace pro DNS záznamy očekávaných typů.

Kapitola 2

Spuštění programu

Pro spuštění programu je třeba ho nejprve zkompilovat na koncovém zařízení. Jedním z požadavků projektu byla jeho přeložitelnost na referenčních serverech *Merlin* a *Eva*.

Pro jednoduchou kompilaci se ve složce projektu nachází soubor `Makefile`, který zavoláním příkazu v terminále

```
$ make
```

provede překlad zdrojových souborů a vytvoří spustitelný soubor `dns_monitor`.

Program lze spustit zavoláním

```
$ ./dns-monitor (-i <interface> | -p <pcapfile>) [-v] [-d <dfile>] [-t <tfile>]
```

Je možné, že při pokusu o spuštění s přepínačem `-i` dojde k selhání z důvodu, že programu chybí oprávnění pro přístup k datům ze síťové karty. Práva lze programu přidělit pomocí příkazu `sudo` zavoláním

```
$ sudo ./dns-monitor -i <interface> [-v] [-d <dfile>] [-t <tfile>]
```

Kde jednotlivé přepínače znamenají toto:

`-i <interface>` *program bude poslouchat na síťovém rozhraní* `<interface>`,
`-p <pcapfile>` *program zanalyzuje dns komunikaci z pcap souboru* `<pcapfile>`,
`-v` *na výstup budou vypisována data podrobněji, komplexní výpis* ,
`-d <dfile>` *do souboru* `<dfile>` *budou zapisována doménová jména obdržených záznamů*,
`-t <tfile>` *do souboru* `<tfile>` *bodou zapisovány překlady doménových jmen.*

Požadavek zadání byl aby u přepínačů `-d` a `-t` byl soubor jako výstupní. Ve své implementaci jsem toto chování rozšířil tak, že se soubor použije i jako vstup, tedy program si ze souboru načte záznamy, které si uloží do struktury lineárního seznamu (datové struktury definované v souboru `file_records_mgmt.h`) a pak už je považuje za znalé.

Při spuštění na síťovém rozhraní program vyžaduje, aby síťové rozhraní na linkové vrstvě komunikovalo pomocí ethernetových hlaviček (`DLT_EN10MB`), případně pomocí Linux Cooked-mode Capture hlaviček (`DLT_LINUX_SLL`, takové využívá např. síťové rozhraní `any`).

Ostatní typy nejsou podporovány a při zachycení nepodporované hlavičky se program ukončí s chybovou hláškou.

Kapitola 3

Použité knihovny

Program užívá celou řadu standardních knihoven, jejichž výčet by bylo rozumnější zjistit nahlédnutím do zdrojových souborů. Za zmínku stojí to, že při použití zpracování argumentů pomocí knihovny `getopt.h` jsem nebyl spokojený s tím, jak se knihovna chová v případě, že povinný argument chybí, proto jsem v projektu použil makro pro zpracování volitelných argumentů, které jsem aplikoval na povinné argumenty (ano je to trochu obcházení problému, ale aspoň pro můj účel teď chování chodu programu dává smysl). [7]

Pro chod programu jsou esenciální knihovny:
`pcap.h` *poskytuje rozhraní pro odposlech síťové komunikace na vybraném rozhraní*,
`resolv.h` *poskytuje funkce a makra pro zpracování obsahu dns zpráv a zdrojových záznamů.*
Práce s oběma knihovnami byla značně usnadněna použitím jejich dokumentace. [2] [3]

A dále mnou napsané knihovny:
`arg_parser.*` *zpracování argumentů z terminálu*,
`file_records_mgmt.*` *načítaní dat a zápis záznamů do souborů pomocí přepínačů* `-d` *a* `-t`,
`string_t.*` *manipulace s řetězci znaků.*

Knihovna `string_t.*` byla převzata z projektu do předmětu IFJ. Dále řešení v rámci souborů `dns-monitor.*` je nádstavbou nad mnou implementovaný druhý projekt do předmětu IPK, kde jsem implementoval síťový sniffer paketů.

Kapitola 4

DNS komunikace

Domain Name System je globální adresář doménových jmen a síťových služeb, pomocí kterého překládáme doménová jména na IP adresy, to primárně z toho důvodu, že např. `google.com.` se pamatuje mnohem lépe než `142.250.191.78` a to se ještě nebavíme o IPv6 adresách. DNS je implementováno jako dotazovací služba, kde se klient zeptá (*DNS Query*) svého nakonfigurovaného DNS serveru na překlad doménového jména. Server na dotaz buď odpoví (*DNS Answer*) adresou, která je překladem doménového jména, nebo uživateli předá adresu jiného DNS serveru, který by překlad znát mohl. To je dáno tím, že doménová jména internetu tvoří stromovou strukturu, kde každý segment oddělený tečkou odpovídá nějaké větvi. [4]

Ze zadání projektu bylo třeba zpracovat výčet typů DNS záznamů, kde si každý trochu přiblížíme.

A

Dotaz typu *A* obsahuje doménové jméno, třídu komunikace (jiná třída než `IN` se však nepoužívá) a požaduje jeho překlad na IPv4 adresu. Odpověď pak obsahuje TTL (*time to live* - doba v sekundách, po kterou lze překlad považovat za platný), třídu, doménové jméno, tak i jeho překlad. [5]

```
[Question Section]
fit.vut.cz. IN A
[Answer Section]
fit.vut.cz. 14400 IN A 147.229.9.65
```

AAAA

Typ *AAAA* je velice podobný typu *A*, jediný rozdíl je, že doménové jméno se nepřekládá na IPv4, ale na IPv6 adresu. [6]

```
[Question Section]
fit.vut.cz. IN AAAA
[Answer Section]
fit.vut.cz. 14400 IN AAAA 2001:67c:1220:809::93e5:941
```

NS

Pomocí záznamu typu *NS* (zkratka z *nameserver*) zjistíme, které DNS servery jsou pro hledanou doménu autoritativní, tedy který DNS server má doopravdy překlady na IP adresy dané domény. [5]

```
[Question Section]
fit.vut.cz. IN NS
[Answer Section]
fit.vut.cz. 14400 IN NS guta.fit.vutbr.cz.
fit.vut.cz. 14400 IN NS gate.feec.vutbr.cz.
fit.vut.cz. 14400 IN NS rhino.cis.vutbr.cz.
fit.vut.cz. 14400 IN NS kazi.fit.vutbr.cz.
```

MX

Záznam typu *MX* (*mail exchange*) nám říká, na jaké adrese domény fungují její mailservery, protože těchto doménových jmen může být více, je pro každé doménové jméno vyhrazena položka priority, kde nejnižší hodnota značí to, že daný server je nejvíce preferován. [5]

```
[Question Section]
fit.vut.cz. IN MX
[Answer Section]
fit.vut.cz. 14400 IN MX 10 kazi.fit.vutbr.cz.
```

SOA

Záznam typu *SOA* (*start of authority*) obsahuje důležité informace o doméně/zóně, jako primární DNS server, emailovou adresu správce, sekvenční číslo poslední aktualizace a dále časové údaje pro obnovení. [5]

```
[Question Section]
fit.vut.cz. IN SOA
[Answer Section]
fit.vut.cz. 14400 IN SOA guta.fit.vutbr.cz.
                         michal.fit.vut.cz.
                         202411154
                         10800
                         3600
                         691200
                         86400
```

CNAME

Díky záznamu typu *CNAME* (*canonical name*) můžeme získat překlad aliasní domény na kanonické doménové jméno. [5]

```
[Question Section]
www.dnsimple.com. IN CNAME
```

```
[Answer Section]
www.dnsimple.com. 145 IN CNAME dnsimple.com.
```

SRV

Typ *SRV* (*service*) nám umožňuje dotázat se zda na daném serveru funguje nějaká služba. V rámci odpovědi dostaneme doménová jména, před které je předřazeno jméno služby a transportní protokol, na němž je služba prováděna. Dále odpověď obsahuje prioritu (nižší hodnota znamená vyšší prioritu), relativní váhu (pro záznamy s totožnou prioritou, vyšší váha znamená vyšší prioritu), číslo portu na němž služba funguje a kanonické jméno stroje, který službu poskytuje. [5]

```
[Question Section]
_sip._udp.cesnet.cz. IN SRV
[Answer Section]
_sip._udp.cesnet.cz. 3570 IN SRV 100
                                 10
                                 5060
                                 cyrus.cesnet.cz.
```

Kapitola 5

Výpis programu

Program umožňuje 2 způsoby výpisu informací.

Základní výpis

V základním režimu výpisu, tedy pokud není použit přepínač `-v` vypadá výpis přibližně takto:

```
2024-11-15 20:36:28 127.0.0.1 -> 127.0.0.53 (Q 1/0/0/1)
2024-11-15 20:36:28 127.0.0.53 -> 127.0.0.1 (R 1/1/0/1)
```

Výpis lze abstrahovat jako:

```
YYYY-MM-DD HH:mm:SS SrcIp -> DstIp (IsQuery QCnt/AnsCnt/AuthCnt/AddCnt)
```

Jednotlivé segmenty lze chápat takto:

`YYYY-MM-DD`	*datum odeslání paketu*,
`HH-mm-SS`	*čas odeslání paketu*,
`SrcIp`	*zdrojová IP adresa*,
`DstIp`	*cílová IP adresa*,
`IsQuery`	*zda se jedná o dotaz (Q) nebo odpověď (R)*,
`QCnt`	*počet záznamů v sekci dotazů*,
`AnsCnt`	*počet záznamů v sekci odpovědí*,
`AuthCnt`	*počet záznamů v autoritativní sekci*,
`AddCnt`	*počet záznamů v sekci rozšíření.*

Komplexní výpis

Řádky jednotlivých vypsaných informací jsou doplněny popiskem, takže je asi není potřeba tolik rozebírat.

Rozebereme si hlavní rozdíl mezi základním a komplexním výpisem, tím bude to, že základní výpis vypisuje počty záznamů v jednotlivých sekcích bez ohledu na to, zda jsou podporované nebo ne.

A zatímco komplexní výpis počty v sekcích nevypisuje, tak místo toho tiskne záznamy v sekcích, tedy pokud sekce obsahuje záznamy, ale ani jeden z nich není programem podporovaný, bude vytištěno návěští sekce bez následujících záznamů, viz [*AdditionalSection*] dotazu (QR je 0) je prázdná, stejně tak i [*AdditionalSection*] odpovědi (QR je 1) je prázdná.

```
Timestamp: 2024-11-15 20:49:52
SrcIP: 127.0.0.1
DstIP: 127.0.0.53
SrcPort: UDP/56658
DstPort: UDP/53
Identifier: 0xb8be
Flags: QR=0, OPCODE=0, AA=0, TC=0, RD=1, RA=0, AD=1, CD=0, RCODE=0

[Question Section]
fit.vut.cz. IN A

[Additional Section]
====================
Timestamp: 2024-11-15 20:49:52
SrcIP: 127.0.0.53
DstIP: 127.0.0.1
SrcPort: UDP/53
DstPort: UDP/56658
Identifier: 0xb8be
Flags: QR=1, OPCODE=0, AA=0, TC=0, RD=1, RA=1, AD=0, CD=0, RCODE=0

[Question Section]
fit.vut.cz. IN A

[Answer Section]
fit.vut.cz. 3319 IN A 147.229.9.65

[Additional Section]
====================
```

Dále rozšířený výpis obsahuje příznaky DNS zprávy (*FLAGS*). [1] [5]

`QR`	0 *pokud se jedná o dotaz,* 1 *pokud o odpověď*,
`OPCODE`	*určení typu dotazu*,
`AA`	*autoritativní odpověď*,
`TC`	*příznak toho, zda byla zpráva zkrácena*,
`RD`	*odmítnutí rekurze*,
`RA`	*dostupnost rekurze*,
`AD`	*potvrzení autenticity dat*,
`CD`	*zamítnutí kontroly autenticity dat*,
`RCODE`	*návratový kód.*

Kapitola 6

Testování

Pro testování chodu programu nemám napsané žádné automatizovatelné jednotkové testy. Testování jsem prováděl nad přiloženými *pcap* soubory v adresáři `/tests`, z nichž získaný výstup jsem pak porovnával s vizualizovanými daty v programu **Wireshark**. Nápodobně jsem program testoval i při poslechu na síťových rozhraních.

Dále jsem také produkt testoval na referenčních serverech, kde byl záměr zjistit, že se přeložený program chová totožně jako ve vývojovém prostředí.

Kapitola 7

Závěr

V rámci časového výseku se mi úspěšně podařilo implementovat produkt. Bylo zajímavé zjistit jak funguje směrování internetu na bázi DNS komunikace, protože nejsem osoba, která často zabíhá do problematik sítí, byla to zajímavá osvěta.

Projekt se mi poměrně líbil, i když musím uznat, že sjednocení duality referenčních serverů, kde se očekávaná konzistence chování mezi systémem unixu a freebsd nemůže více rozcházet, je pak nepříjemnost vymýšlet řešení pro obcházení problému, viz. pravidlo `all` v souboru `Makefile`.

Literatura

[1] Eastlake, D. a Kaufman, C. *RFC2065: Domain Name System Security Extensions.* RFC Editor, 1997.

[2] Jacobson, V.; Leres, C. a McCanne, S. *Pcap(3PCAP) man page* online. Dostupné z: https://www.tcpdump.org/manpages/pcap.3pcap.html. [cit. 07-10-2024].

[3] Liu, C. a Albitz, P. *Programming with the Resolver Library Routines* online. Dostupné z: https://web.deu.edu.tr/doc/oreily/networking/dnsbind/ch14_02.htm. [cit. 08-10-2024].

[4] Matoušek, P. Systém DNS. In: *Síťové aplikace a správa sítí.* 2024.

[5] Mockapetris, P. V. *RFC1035: Domain names - implementation and specification.* RFC Editor, 1987.

[6] Thomson, S.; Huitema, C.; Ksinant, V. a Souissi, M. *RFC3596: DNS Extensions to Support IP Version 6.* RFC Editor, 2003.

[7] Wik, L. E. *Optional arguments with getopt_long(3)* online. 2021. Dostupné z: https://cfengine.com/blog/2021/optional-arguments-with-getopt-long/. [cit. 26-09-2024].